



Selected Work

Three ways I approach a problem: commercially, operationally, and in code.

I'm an IT professional who's passionate about music, TV, and other media. This piece shows how I approach a problem in code: a real, runnable Python and SQL prototype that combines multi-source ratings data into one trustworthy score.

Technical prototype

TV Show Combined Ratings Aggregator

A real, runnable Python and SQL prototype - synthetic episode ratings pulled from three review sources (IMDB, Rotten Tomatoes, Metacritic), each on a different scale, combined into one overall score per episode and per season.

444

score submissions

144

episodes scored

193

quality issues caught

2

engines: pandas + SQL

- IMDB rates 0-10, Rotten Tomatoes and Metacritic rate 0-100 - every score gets converted onto one common scale before anything is averaged, or IMDB would be badly under-weighted
- A missing or bad source doesn't sink the whole episode - the combined score uses whichever sources are valid, and says plainly how many of the 3 it's based on
- Show titles resolve against a small reference crosswalk (canonical ID to known aliases) - built after catching a real bug where inconsistent spelling was silently splitting one episode into several

Normalize, then combine - Python/pandas

```
# IMDB is 0-10, the other two are already 0-100 -
# converting onto one scale before averaging is the whole point
def normalize(row):
    if row["source"] == "imdb":
        return row["raw_score_num"] * 10
    return row["raw_score_num"]

clean["normalized_score"] = clean.apply(normalize, axis=1)

episode_scores = (
    clean.groupby(["episode_id", "canonical_title", "season", "episode"])
    .agg(combined_score=("normalized_score", "mean"),
         sources_used=("source", "nunique"))
)
```

Same logic, standalone SQL

```
-- Combined score per episode, from a view that already
-- normalized scales and resolved canonical titles
SELECT episode_id, canonical_title, season, episode,
       ROUND(AVG(normalized_score), 1) AS combined_score,
```

```

COUNT(*) AS sources_used
FROM view_normalized
GROUP BY episode_id, canonical_title, season, episode
ORDER BY episode_id;

```

Run output - pandas engine

```

Total score submissions processed: 444
Records with an exception:        176
Total exceptions raised:          193

Exceptions by rule:
INCONSISTENT_SHOW_TITLE          148
DUPLICATE_SUBMISSION             24
MISSING_OR_INVALID_SCORE         11
SCORE_OUT_OF_RANGE               10

Episodes with a combined score: 144
...of which based on fewer than 3 sources: 19
Seasons scored: 18

```

Run output - SQL engine (cross-checked against pandas)

B99-S1E1	Brooklyn Nine-Nine	combined_score=74.7	sources_used=3
B99-S1E2	Brooklyn Nine-Nine	combined_score=65.3	sources_used=3
B99-S1E3	Brooklyn Nine-Nine	combined_score=75.3	sources_used=3
B99-S1E4	Brooklyn Nine-Nine	combined_score=87.5	sources_used=2
Breaking Bad	Season 1	season_score=78.9	episodes_scored=8
Breaking Bad	Season 2	season_score=79.5	episodes_scored=8
Brooklyn Nine-Nine	Season 1	season_score=77.6	episodes_scored=8

Files

generate_ratings.py - synthetic multi-source score generator
 show_reference.csv - show-ID crosswalk (title aliases to canonical ID)
 validate_ratings.py - Python/pandas quality checks + score aggregation
 load_to_sqlite.py - loads the data into a local SQLite database
 reconcile.sql - standalone SQL quality checks + score aggregation
 run_sql_reconciliation.py - sets up views, runs the SQL, prints results
 README.md - design notes, a bug I found and fixed, and how to run it

In a production system, this same pattern - validate, normalize onto a common scale, combine, and stay transparent about data coverage - is exactly how I handle data quality and reconciliation work in my current role, just against fund and reference data instead of episode ratings.

All company names, products, and datasets above are fictional or synthetic, built to demonstrate approach rather than represent real work product.

Gur Sabharwal · gursabharwal01@gmail.com · linkedin.com/in/gursabharwal01